

Documentation technique

Architecture, installation et exploitation

Version 2.0.4 - exemples anonymisés

1. Objectif

Ce document décrit l'installation, la configuration Apache, le comportement réseau et les tests d'exploitation de Trax ILIAS Bridge 2.0.4. Il s'adresse aux administrateurs système et aux exploitants. Les exemples utilisent uniquement des valeurs génériques.

2. Architecture logique

L'adaptateur est un contrôleur PHP appelé par Apache via AliasMatch avant que certaines requêtes n'atteignent Trax.



3. Routes interceptées

Route	Méthodes	Traitement
/cmi5/tokens	POST	Suppression de activity_id et proxy vers Trax.
/xapi/activities/state	GET, PUT, POST, DELETE selon usage	Suppression de activity_id et proxy vers Trax.
/api/statements/aggregate avec store	GET	Lecture des statements xAPI et agrégation compatible ILIAS.
/api/statements/aggregate sans store	GET	Résolution du store puis agrégation compatible ILIAS.

4. Installation des fichiers

```

mkdir -p /var/www/trax-ilias-aggregate-adapter
cp aggregate.php /var/www/trax-ilias-aggregate-adapter/aggregate.php
cp config.sample.php /var/www/trax-ilias-aggregate-adapter/config.php
chown -R apache:apache /var/www/trax-ilias-aggregate-adapter
chmod 0644 /var/www/trax-ilias-aggregate-adapter/aggregate.php
chmod 0640 /var/www/trax-ilias-aggregate-adapter/config.php

```

Exemples génériques uniquement : trax.example.org / ilias.example.org

Vérification de la syntaxe PHP :

```
php -l /var/www/trax-ilias-aggregate-adapter/aggregate.php
php -l /var/www/trax-ilias-aggregate-adapter/config.php
```

5. Configuration Apache simple HTTP/IP

Pour un environnement direct sans reverse proxy, le fichier simple peut être copié dans conf.d :

```
cp apache-trax-ilias-adapter.conf /etc/httpd/conf.d/trax-ilias-aggregate-adapter.conf
apachectl configtest
systemctl reload httpd
```

Attention : cette configuration simple place les AliasMatch globalement. Elle ne doit pas être utilisée telle quelle si l'adaptateur doit rappeler Trax par le même Apache et le même chemin intercepté.

6. Configuration Apache HTTPS avec nom public et backend interne

Dans une installation HTTPS avec un nom public, l'adaptateur ne doit pas rappeler l'URL publique. Sinon, Apache réintercepte la requête et l'adaptateur peut boucler sur lui-même. La solution recommandée consiste à exposer Trax publiquement sur HTTPS, tout en créant un vhost interne local qui sert Trax directement sans AliasMatch.

6.1 Exemple complet

```
Listen 127.0.0.1:8080

<VirtualHost *:443>
    ServerName trax.example.org
    DocumentRoot /Data/www/traxlrs/public

    SSLEngine on
    SSLCertificateFile /etc/pki/tls/certs/trax.example.org.crt
    SSLCertificateKeyFile /etc/pki/tls/private/trax.example.org.key
    SSLCertificateChainFile /etc/httpd/ssl/ca/example-chain.crt

    AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/api/statements/aggregate$" "/var/www/trax-ilias-
aggregate-adapter/aggregate.php"
    AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/api/statements/aggregate$" "/var/www/trax-
ilias-aggregate-adapter/aggregate.php"
    AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/cmi5/tokens$" "/var/www/trax-ilias-
aggregate-adapter/aggregate.php"
    AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/xapi/activities/state$" "/var/www/trax-
ilias-aggregate-adapter/aggregate.php"

    <Directory "/var/www/trax-ilias-aggregate-adapter">
        Require all granted
        Options -Indexes
        AllowOverride None
    </Directory>

    <Directory /Data/www/traxlrs/public>
        Options Indexes FollowSymLinks MultiViews
        AllowOverride All
        Require all granted
    </Directory>

    SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1
```

```

    ErrorLog /var/log/httpd/trax-ssl_error.log
    CustomLog /var/log/httpd/trax-ssl_access.log combined
</VirtualHost>

<VirtualHost 127.0.0.1:8080>
    ServerName trax-internal.local
    DocumentRoot /Data/www/traxlrs/public

    <Directory /Data/www/traxlrs/public>
        Options Indexes FollowSymLinks MultiViews
        AllowOverride All
        Require all granted
    </Directory>

    SetEnvIf Authorization "(.*)" HTTP_AUTHORIZATION=$1

    ErrorLog /var/log/httpd/trax-internal_error.log
    CustomLog /var/log/httpd/trax-internal_access.log combined
</VirtualHost>

<VirtualHost *:80>
    ServerName trax.example.org
    Redirect permanent / https://trax.example.org/
</VirtualHost>

```

6.2 Points clés

- Les AliasMatch doivent être dans le vhost public HTTPS uniquement.
- Le vhost interne 127.0.0.1:8080 ne contient aucun AliasMatch de l'adaptateur.
- config.php doit pointer vers http://127.0.0.1:8080.
- Les logs publics et internes doivent être séparés pour faciliter le diagnostic.

7. Configuration applicative config.php

```

<?php
return [
    'trax_base_url' => 'http://127.0.0.1:8080',
    'aggregate_store_strategy' => 'client_name',
    'client_store_map' => [
        'eform' => 'default',
    ],
    'default_store' => 'default',
    'max_statements_to_fetch' => 5000,
    'debug' => false,
];

```

Clé	Rôle
trax_base_url	URL utilisée par l'adaptateur pour appeler Trax. En HTTPS avec nom public, préférer un backend interne.
aggregate_store_strategy	Stratégie de résolution du store quand ILIAS omet le store dans l'URL aggregate.
client_store_map	Correspondance client Trax vers store Trax. Indispensable si client et store ont des noms différents.
max_statements_to_fetch	Nombre maximal de statements chargés pour construire la réponse aggregate.
debug	Active les erreurs détaillées en JSON ; à laisser false en production.

8. Tests techniques

Tester le backend Trax interne :

```
curl -i "http://127.0.0.1:8080/trax/api/gateway/clients/eform/stores/default/xapi/about"
```

Réponse attendue :

```
HTTP/1.1 200 OK
{"version":["1.0.3"]}
```

Tester la route publique interceptée. Un GET sur cmi5/tokens doit retourner 405, car la route attend POST :

```
curl -k -i
"https://trax.example.org/trax/api/gateway/clients/eform/stores/default/cmi5/tokens?activity_id=test"
```

Tester les logs pendant un lancement cmi5 :

```
tail -f /var/log/httpd/trax-ssl_access.log /var/log/httpd/trax-ssl_error.log
/var/log/httpd/trax-internal_access.log /var/log/httpd/trax-internal_error.log | grep --line-
buffered -E "cmi5/tokens|activities/state|LMS.LaunchData|xapi/statements|aggregate| 400 | 403 | 404 | 500
| 504"
```

9. Résultats attendus dans les logs

Flux	Code attendu	Signification
PUT LMS.LaunchData	204	Les données de lancement cmi5 sont écrites.
GET LMS.LaunchData	200	Les données de lancement sont relues.
POST /xapi/statements	200 ou 204	Le contenu écrit ses statements.
GET /api/statements/aggregate	200	ILIAS reçoit une réponse compatible pour ses onglets.
POST /cmi5/tokens	200/204 selon Trax	Le token cmi5 peut être créé.

10. Erreurs fréquentes et diagnostic

Symptôme	Cause probable	Correction
500/504 répétés depuis 127.0.0.1	Boucle adaptateur vers lui-même	Créer un vhost interne sans AliasMatch et configurer trax_base_url.
403 cmi5-tokens/write	Capacité Trax manquante	Ajouter cmi5-tokens/write au client Trax.
PUT LMS.LaunchData en 400	Paramètre activity_id non standard sur State API	Activer la route /xapi/activities/state dans l'adaptateur.
Ranking vide	Absence de result.score.scaled sur l'activité principale	Configurer le contenu pour envoyer un score global.
Erreur ILIAS getTimestamp() on false	Timestamp absent ou non parsable	Utiliser la normalisation 2.0.4 et vérifier les statements de test.

11. Sécurité et exploitation

- Limiter l'accès au vhost interne à 127.0.0.1.
- Ne pas activer debug en production.
- S'assurer que les droits fichiers empêchent la lecture non souhaitée de config.php.
- Conserver des logs distincts pour le public et l'interne.
- Vérifier régulièrement que les routes xAPI standards non-cmi5 ne sont pas interceptées.